Formal Review Request Template

Requester: _____Alvin Mawlue_____
Reviewer: Vedaant Patel_____
Date: _____10/29/2026_____

Approx. review duration: ___1_____

Choose **one primary review method**:

Code / Software Review

- ☑ **Code Quality Review**
  Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.
- ☐ **White-Box / Edge-Case Testing**
  Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.
- ☐ **Black-Box / Functional Testing**
  Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.
- ☐ **Performance / Runtime Testing**
  Systematically evaluate whether the software meets relevant runtime or performance expectations.

Usability Review

- ☐ **Task-Based Usability Testing**
  Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.
- ☐ **Heuristic / Guidelines Evaluation**
  Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.
- ☐ **Prototype Usability Testing**
  Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

Design Review

- ☐ **Architecture / Conceptual Design Review**
  Evaluate whether architecture and major design decisions align with project requirements and constraints.
- ☐ **Preliminary Design / UML Review**
  Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.
- ☐ **Security Design Review**
  Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

Other

- ☐ **Other review method:**
  _____
  _____

Explain what the reviewer will evaluate and why this method is appropriate for your work:

Review the readability and maintainability of our codes. This method is appropriate because we turning over the codes to a different group after this semester

---

Review Request Details

1) What specifically should the reviewer examine or interact with?

The models from the different applications

---

2) What question do you want this review to help answer?

Examples:

- Does this implementation correctly handle boundary and error cases?
- Can a new user complete the checkout task without assistance?
- Does this architecture satisfy our project's stated requirements?

---

3) What should the reviewer focus on?

---

4) What should the reviewer *not* evaluate during this review?

Examples:

- Do not evaluate visual polish or mobile responsiveness (for a usability test evaluating navigation and task completion)
- Do not evaluate the database layer or unfinished API integration (for a code review focusing on the authentication module)

View page

---

5) What should the reviewer use to judge the work?

Examples include expected software behavior, project requirements, coding best practices, usability heuristics, or design constraints.

Project requirements

Reviewer Section

1) Briefly describe what you actually did during the review.

-I reviewed the Django models to understand how surveys, assessments, and responses are created and connected. I inspected the field definitions, database relationships, validation rules, and calculation methods. I also looked for potential issues with data integrity, missing validation, and how the models handle invalid or incomplete information.

---

2) What portions of the artifact did you evaluate?

-I evaluated the following portions of the code:

- Survey model: Reviewed the title field and question-related string representation.
- TrendDirection model: Reviewed the available trend choices (Improving, Stable, Declined).
- Assessment model: Evaluated user and survey relationships, assessment fields, scoring fields, and average calculation method.
- Response model: Reviewed assessment and question relationships, response value validation, and the unique constraint preventing duplicate responses.

---

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.

Reviewed the provided Django model code, focusing on model relationships, field definitions, validation rules, and database constraints. Identified potential issues and documented findings with specific examples from the code.

Add Assessment and Response models (#17) by insp-dpl · Pull Request #20 · memphis-iis/farmer-health · GitHub

---

Findings

Strengths:

- The models use appropriate Django relationships, including ForeignKey relationships, to connect users, surveys, assessments, and responses. The use of `on_delete` behaviors also helps protect important records.
- 2. The Response model includes minimum and maximum value validators to keep

responses within the expected 1–5 range. It also uses a UniqueConstraint to prevent users from submitting multiple responses to the same question within one assessment.

Issues / Defects:

Description of Issue	Severity (Low/Med/High)	Evidence / Example	Suggestions
Response values may not be validated when records are saved directly through the model.	Med	The `PositiveIntegerField` and validators enforce expected values during validation, but Django does not automatically run validators on every `.save()` call	Consider overriding `clean()` or ensuring `full_clean()` is called before saving responses.
The average calculation does not handle assessments with no responses.	Low	`calculate_average()` returns the result of `Avg("value")`, which will be None if there are no responses.	Add handling for empty response sets so the application can handle missing averages safely.
Assessment scoring fields can potentially become inconsistent with the associated responses.	Med	Fields such as `raw_total`, `average_score`, `count_low_1_2`, and `weighted_score` are stored separately from the Response records.	Ensure these fields are recalculated whenever responses are added, updated, or removed.

Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.